

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-abee8a514f178d541.jsonl`
- SHA-256: `31783c022e761180f03ad77efaebdc020e1c074c8ebbd7f0c99fd3654aec0a7a`
- Source modified: `2026-07-04T00:49:26+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-04T00:48:33.109Z)

Code-review VERIFIER. Repo: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. PR #458 branch = origin/codex/a7-a9-quality-chain (read files via `git show origin/codex/a7-a9-quality-chain:<path>`; PR-branch fusion_golden.py also at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458_fusion_golden.py).

CANDIDATE: PR #458 adds `is\_full\_fusion\_feature\_file(path)` in backend/eval/fusion_golden.py. Its logic: load JSON, get candidates; `if not candidates: return True`; else `return all(isinstance(c,dict) and "shuttle" in c and "person" in c for c in candidates)`.

The repo ALREADY has `backend/eval/experiment.py::is\_fusion\_schema(candidates)` = `return bool(candidates) and all("shuttle" in c and "person" in c for c in candidates)` ? documented as the "Single source of truth for the three loaders (consolidated 2026-06-18 after PRs #209/#210 each landed an independent copy)", and its docstring explicitly says "An empty candidate list is not a fusion file either." So the existing SSOT returns FALSE for empty candidates; the PR's new function returns TRUE.

CLAIM: (a) reuse violation ? the PR adds a THIRD independent copy of a check that has a documented SSOT built specifically to stop this fragmentation; (b) correctness ? the two disagree on the empty-candidates case, and this disagreement causes a silent corpus gap.

VERIFY the correctness impact concretely:

1. In fusion_golden.run(), when a video's existing `\*\_golden\_features.json` has `candidates: []`, `is\_full\_fusion\_feature\_file` returns True ? the resume logic treats it as full-fusion and SKIPS re-extraction (appends `out`, continues). Quote the run() lines.
2. Meanwhile, the downstream loaders (ablation.load_videos, fusion_compare.load_videos, fusion_audio.load_videos_with_audio) use `is\_fusion\_schema` (empty ? False) ? they SKIP that video as windowing-only/non-fusion. Confirm which schema check those loaders use (grep them).
3. So a video that extracted to 0 candidate windows is (a) never re-extracted by run() resume AND (b) excluded from every fusion analysis ? a silent gap. Is a 0-candidate extraction reachable? (A video with no detected rally windows, or a very short/empty clip. The A7 doc shows `mahadevpura\_1` produced just 1 window, so low/zero-window videos exist.)
4. Also: does extract_video ever WRITE candidates:[] ? i.e., is the empty-file state produced by this code path (a legit empty extraction) or only by a crash? Check extract_video's write.

Return exactly one verdict: CONFIRMED (name the inputs/state + the wrong outcome, quote the contradicting lines), PLAUSIBLE (mechanism real, trigger uncertain), or REFUTED (e.g. run() never skips empty files, or the loaders use the same new function). Final message: verdict word, 3-6 sentences with quoted lines, then a one-line fix (e.g. "reuse experiment.is_fusion_schema; drop the empty?True special case").

2. assistant (2026-07-04T00:48:34.964Z)

I'll verify this correctness claim step by step. Let me start by examining the code in the PR and the existing SSOT.

3. user (2026-07-04T00:48:35.301Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17
18     from __future__ import annotations
19
20     import argparse
21     import json
22     import os
23     import time
24     from typing import Any, Dict, List, Optional, Sequence
25
26     from backend.eval import distill_local
27     from backend.eval.calibrate_wasb import load_golden_gts
28     from backend.eval.golden_manifest import load_resolved_manifest
29     from backend.eval.training import label_candidates
30     from backend.pipeline.detectors import fusion_features as ff
31     from backend.pipeline.detectors import rally_gate
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33     from backend.utils.run_telemetry import RunTelemetry
34
35     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39     SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
40
41
42     def is_full_fusion_feature_file(path: str) -> bool:
43         """True when an existing golden feature JSON already has shuttle+person cue dicts.
44
45         Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
pattern.
46         Those are useful for gts/windowing analysis, but they are not the replayable fusion
47         substrate that ``fusion_compare`` and ``ablation`` need.
48         """
49
50         try:
51             with open(path, encoding="utf-8") as f:
52                 data = json.load(f)
53         except (OSError, json.JSONDecodeError, TypeError):
54             return False
```

```

55     candidates = data.get("candidates")
56     if not isinstance(candidates, list):
57         return False
58     if not candidates:
59         return True
60     return all(
61         isinstance(candidate, dict)
62         and "shuttle" in candidate
63         and "person" in candidate
64         for candidate in candidates
65     )
66
67
68 def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69     """Load ``video_id`` -> local_path`` from the collector golden-source manifest.
70
71     The collector records source/proxy ownership and full-file MD5s. Only entries marked
72     ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73     """
74
75     if not source_manifest_path:
76         return {}
77     with open(source_manifest_path, encoding="utf-8") as f:
78         data = json.load(f)
79     entries = data.get("videos", data) if isinstance(data, dict) else data
80     if not isinstance(entries, list):
81         raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
82
83     paths: Dict[str, str] = {}
84     for entry in entries:
85         if not isinstance(entry, dict):
86             continue
87         video_id = entry.get("video_id") or entry.get("name")
88         local_path = entry.get("local_path")
89         if not video_id or not local_path:
90             continue
91         if entry.get("source_status", "present_local") != "present_local":
92             continue
93         paths[str(video_id)] = str(local_path)
94     return paths
95
96
97 def attach_source_video_paths(
98     specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99 ) -> int:
100     """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101
102     Returns the number of specs updated. The collector source manifest is authoritative
103     for local source/proxy video location and overrides derived/stale manifest paths.
104     """
105
106     source_paths = load_source_video_paths(source_manifest_path)
107     updated = 0
108     for spec in specs:
109         name = str(spec.get("name", ""))
110         source_path = source_paths.get(name)
111         if not source_path:
112             continue
113         if spec.get("video_path") == source_path:
114             continue
115         spec["video_path"] = source_path
116         updated += 1
117     return updated
118

```

```

119
120 def _filter_specs(
121     specs: Sequence[Dict[str, Any]], only: Sequence[str]
122 ) -> List[Dict[str, Any]]:
123     if not only:
124         return list(specs)
125     wanted = list(dict.fromkeys(only))
126     wanted_set = set(wanted)
127     filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128     found = {str(spec.get("name")) for spec in filtered}
129     missing = [name for name in wanted if name not in found]
130     if missing:
131         raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
132     return filtered
133
134
135 def _derive_video_path(spec: Dict[str, Any]) -> str:
136     """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
137     golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
138     convention)."""
139     if spec.get("video_path"):
140         return str(spec["video_path"])
141     labels = str(spec["labels"])
142     if labels.endswith(".rallies.csv"):
143         return labels[: -len(".rallies.csv")] + ".MP4"
144     raise ValueError(
145         f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
146     )
147
148 def extract_video(
149     spec: Dict[str, Any],
150     detector: Optional[Any] = None,
151     frame_skip: int = 3,
152     iou: float = 0.5,
153     log_every_sec: float = 15.0,
154     telemetry: Optional[Any] = None,
155     use_gpu: bool = True,
156     detector_model: Optional[str] = None,
157     max_gpu_temp_c: Optional[float] = None,
158 ) -> Dict[str, Any]:
159     """One golden video -> a feature record (shuttle + person features per candidate
160     window).
161     Returns `{name, fps, frame_width,
162     candidates: [{start, end, shuttle{ }, person{ }, label}], gts}``.
163     `detector` (anything with `detections_over_windows`) is injectable for GPU-free
164     tests;
165     when None a real `PersonDetector` is built (loads torchvision on first use).
166     `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
167     snapshots
168     from the detection progress callback so a hard kill leaves a forensic runlog.
169     """
170     fps, fw = float(spec["fps"]), float(spec["frame_width"])
171     traj = str(spec["trajectory"])
172     points = TrackNetRunner.parse_trajectory_csv(traj)
173     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
174     gts = load_golden_gts(str(spec["labels"]))
175     labels = label_candidates(intervals, gts, iou)
176     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
177     estimate
178     # (parity with the shuttle net-crossing gate) rather than hard-coding an
179     orientation.
180     axis, net_px, _span = rally_gate.estimate_play_axis(points)
181     video_path = _derive_video_path(spec)

```

```

177         if not os.path.isfile(video_path):
178             raise FileNotFoundError(
179                 f"{spec.get('name')}: source video not found for fusion extraction:
{video_path}"
180             )
181
182         if detector is None:
183             from backend.pipeline.detectors.person_detector import PersonDetector
184
185             indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
186             if detector_model:
187                 indexing_cfg["detector_model"] = detector_model
188             detector = PersonDetector({"indexing": indexing_cfg})
189
190         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192         n = len(intervals)
193         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194         print(
195             f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
pass...",
196             flush=True,
197         )
198         t0 = time.monotonic()
199         state = {"last": t0}
200
201         def _progress(done: int, total: int) -> None:
202             rec = None
203             if telemetry is not None:
204                 rec = telemetry.snapshot(done, total, phase=spec["name"])
205             if max_gpu_temp_c is not None and rec is not None:
206                 gpu = rec.get("gpu") or {}
207                 temp = gpu.get("temp_c")
208                 if temp is not None and float(temp) >= max_gpu_temp_c:
209                     raise RuntimeError(
210                         f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
211                         f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
212                     )
213             now = time.monotonic()
214             if now - state["last"] >= log_every_sec or done >= total:
215                 el = now - t0
216                 rate = done / el if el > 0 else 0.0
217                 eta = (total - done) / rate if rate > 0 else 0.0
218                 pct = 100 * done // total if total else 0
219                 print(
220                     f"[fusion_golden] {spec['name']}: detect frame {done}/{total} ({pct}%)
"
221                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
222                     flush=True,
223                 )
224                 state["last"] = now
225
226         per_window = detector.detections_over_windows(
227             video_path, win_frames, frame_skip, progress_cb=_progress
228         )
229
230         candidates: List[Dict[str, Any]] = []
231         for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
232             det = per_window[i]
233             det_fw = det.get("frame_width") or fw
234             det_fh = det.get("frame_height") or 0.0
235             # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
236             # normalisation fusion_features uses (x/width, y/height).

```

```

237         if axis == "x":
238             net_norm = net_px / det_fw if det_fw > 0 else 0.5
239         else:
240             net_norm = net_px / det_fh if det_fh > 0 else 0.5
241         sf, ef = win_frames[i]
242         shuttle_in = [p for p in points if sf <= p.frame <= ef]
243         person = ff.compute_person_features(
244             det["frames"],
245             shuttle_in,
246             det_fw,
247             det_fh,
248             court_polygon=None,
249             net=(axis, net_norm),
250         )
251         candidates.append(
252             {
253                 "start": float(s),
254                 "end": float(e),
255                 "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
256                 "person": {k: float(v) for k, v in person.items()},
257                 "label": int(labels[i]),
258             }
259         )
260     return {
261         "name": spec["name"],
262         "fps": fps,
263         "frame_width": fw,
264         "candidates": candidates,
265         "gts": [[float(a), float(b)] for a, b in gts],
266     }
267
268
269 def run(
270     manifest_path: str,
271     out_dir: str,
272     frame_skip: int = 3,
273     iou: float = 0.5,
274     fresh: bool = False,
275     smallest_first: bool = False,
276     source_manifest: Optional[str] = None,
277     only: Sequence[str] = (),
278     missing_fusion_only: bool = False,
279     use_gpu: bool = True,
280     detector_model: Optional[str] = None,
281     max_gpu_temp_c: Optional[float] = None,
282 ) -> List[str]:
283     specs = load_resolved_manifest(manifest_path)
284     source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
285     if source_manifest:
286         updated = attach_source_video_paths(specs, source_manifest)
287         print(
288             f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
289             f"from {source_manifest}",
290             flush=True,
291         )
292     specs = _filter_specs(specs, only)
293     if smallest_first:
294         # Process small videos first for quick feedback (the slow full match lands
295         # last);
296         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
297         # row/frame).
298         specs = sorted(
299             specs,
300             key=lambda s: (
301                 os.path.getsize(s["trajectory"])

```

```

300         if os.path.isfile(s.get("trajectory", ""))
301         else 0
302     ),
303 )
304 os.makedirs(out_dir, exist_ok=True)
305 # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
306 # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
307 # break extraction, so guard the build and fall back to no telemetry on any error.
308 telem: Optional[RunTelemetry]
309 try:
310     telem = RunTelemetry(
311         os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
312     )
313 except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
314     telem = None
315 written: List[str] = []
316 for vi, spec in enumerate(specs, 1):
317     out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
318     existing = os.path.isfile(out)
319     existing_full = existing and is_full_fusion_feature_file(out)
320     # Resumable: skip a video whose full-fusion JSON already exists. A leaner
321     # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
322     if missing_fusion_only and not fresh and existing_full:
323         print(
324             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325             f"features already present -> {out}",
326             flush=True,
327         )
328         written.append(out)
329         continue
330     if not fresh and existing and existing_full:
331         print(
332             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
"
333             f"-> {out} (use --fresh to redo)",
334             flush=True,
335         )
336         written.append(out)
337         continue
338     if not fresh and existing and not existing_full:
339         print(
340             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341             f"is not full-fusion schema; re-extracting...",
342             flush=True,
343         )
344         print(
345             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
346             flush=True,
347         )
348     t0 = time.monotonic()
349     if telem is not None:
350         telem.event("video_start", spec["name"])
351     try:
352         data = extract_video(
353             spec,
354             frame_skip=frame_skip,
355             iou=iou,
356             telemetry=telem,
357             use_gpu=use_gpu,
358             detector_model=detector_model,
359             max_gpu_temp_c=max_gpu_temp_c,
360         )

```

```

361         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
362             if telem is not None:
363                 telem.event("error", spec["name"], err=str(e))
364             raise
365         n_pos = sum(c["label"] for c in data["candidates"])
366         with open(out, "w", encoding="utf-8") as f:
367             json.dump(data, f, indent=2)
368         if telem is not None:
369             telem.event(
370                 "video_done",
371                 spec["name"],
372                 windows=len(data["candidates"]),
373                 positives=n_pos,
374             )
375         print(
376             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378             f"-> {out}",
379             flush=True,
380         )
381         written.append(out)
382     return written
383
384
385 def main() -> None:
386     ap = argparse.ArgumentParser(
387         description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
388     )
389     ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
390     ap.add_argument("--out-dir", default="output")
391     ap.add_argument("--frame-skip", type=int, default=3)
392     ap.add_argument("--iou", type=float, default=0.5)
393     ap.add_argument(
394         "--fresh", action="store_true", help="re-extract even if a JSON already exists"
395     )
396     ap.add_argument(
397         "--smallest-first",
398         action="store_true",
399         help="process smaller videos first for quick feedback (LOVO is order-
independent)",
400     )
401     ap.add_argument(
402         "--source-manifest",
403         default=None,
404         help=(
405             "collector golden_source_videos.json; overrides video paths for source/proxy
"
406             f"videos (or set {SOURCE_MANIFEST_ENV})"
407         ),
408     )
409     ap.add_argument(
410         "--only",
411         action="append",
412         default=[],
413         metavar="VIDEO_ID",
414         help="process one manifest video by name; repeatable",
415     )
416     ap.add_argument(
417         "--missing-fusion-only",
418         action="store_true",
419         help="skip videos whose existing JSON already has full shuttle+person fusion
schema",

```



```

420     )
421     ap.add_argument(
422         "--cpu",
423         action="store_true",
424         help="run torchvision person detection on CPU instead of CUDA",
425     )
426     ap.add_argument(
427         "--detector-model",
428         default=None,
429         choices=[
430             "fasterrcnn_resnet50_fpn",
431             "fasterrcnn_mobilenet_v3_large_fpn",
432             "retinanet_resnet50_fpn",
433         ],
434         help="torchvision detector model; default keeps the configured Faster R-CNN
ResNet",
435     )
436     ap.add_argument(
437         "--max-gpu-temp-c",
438         type=float,
439         default=None,
440         help="abort extraction if telemetry sees GPU temperature at or above this
value",
441     )
442     args = ap.parse_args()
443     run(
444         args.manifest,
445         args.out_dir,
446         args.frame_skip,
447         args.iou,
448         fresh=args.fresh,
449         smallest_first=args.smallest_first,
450         source_manifest=args.source_manifest,
451         only=args.only,
452         missing_fusion_only=args.missing_fusion_only,
453         use_gpu=not args.cpu,
454         detector_model=args.detector_model,
455         max_gpu_temp_c=args.max_gpu_temp_c,
456     )
457
458
459     if __name__ == "__main__":
460         main()
461

```

4. assistant (2026-07-04T00:48:37.223Z)

Now let me check the SSOT function in experiment.py:

5. user (2026-07-04T00:48:38.598Z)

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/experiment.py

6. user (2026-07-04T00:48:40.439Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`          ? `score`, `match_intervals`, `interval_iou`

```

```

10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`    ? `pct_improvement`
15
16 The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17 (per-frame signals ? run once on the GPU/WSL box, persisted into
18 `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19 signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20 given persisted candidate features it produces `List[Interval]` deterministically,
21 with no GPU and zero production risk. So most experiments are pure offline
22 re-scoring of stored data and never touch the served pipeline.
23
24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38
39 from __future__ import annotations
40
41 import glob
42 import json
43 import logging
44 import os
45 from dataclasses import dataclass, field
46 from datetime import datetime, timezone
47 from hashlib import sha1
48 from typing import Any, Callable, Dict, List, Optional, Sequence
49
50 from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
51 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count ratio
52 from backend.eval.calibration import pct_improvement
53 from backend.eval.classifier import LogisticRegression
54 from backend.eval.gemini_refine import merge_overlaps
55 from backend.eval.metrics import Interval, match_intervals, score
56 from backend.eval.regression import gt_hash
57 from backend.eval.score_calibration import ( # C2: calibrate cue scores
58     fit_isotonic,
59     fit_platt,
60 )
61 from backend.eval.training import label_candidates
62
63 logger = logging.getLogger(__name__)
64
65 # Where a comparison run appends one reproducible record. Tracked location (NOT under
66 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
67 # regression.DEFAULT_BASELINE_PATH.
68 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
69 _METRICS = ("precision", "recall", "f1", "mean_iou")
70
71
72 class ExperimentError(ValueError):
73     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
74     score an unlabeled video with no pinned model, or a gate referencing an absent

```

```

75         feature)."""
76
77
78     # ----- Variant
79
80
81     @dataclass
82     class Variant:
83         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
84
85         A variant turns one video's persisted candidate windows + features into rally
86         intervals. Every knob defaults to the control behaviour (no gate, no learned
87         filter), so a variant that merely carries a new, disabled cue is byte-identical
88         to control ? the core no-regression guarantee.
89
90         Fields:
91         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
92           for the leaderboard and for selecting the served path (the existing
93           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
94           New cues are recorded here default-OFF.
95         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
96           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
97           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
98           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
99         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
100          whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
101          cue persists that feature).
102         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
103           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
104           required for ``compare_unlabeled`` on a learned variant.
105         - ``feature_names`` ? the persisted features the learned filter consumes. When set
106           WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
107           (honest) ? the recipe, not a pinned artifact.
108         - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
109           overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
110         """
111
112         name: str
113         segmenter: str = "wasb_hybrid"
114         config_overrides: Dict[str, Any] = field(default_factory=dict)
115         cue_flags: Dict[str, bool] = field(default_factory=dict)
116         min_crossings: int = 0
117         min_players: int = 0
118         classifier_model: Optional[str] = None
119         feature_names: Optional[List[str]] = None
120         threshold: float = 0.5
121         merge_gap: float = 1.0
122         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
123         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
124         # failure where a fixed 0.5 zeroed out a small-candidate video.
125         tune_threshold: bool = False # pick the F1-best threshold on the train fold
126         calibrate: Optional[str] = (
127             None # None | "platt" | "isotonic" ? calibrate proba first
128         )
129
130         def is_learned(self) -> bool:
131             """True if this variant applies a learned classifier (pinned model or
recipe)."""
132             return self.classifier_model is not None or bool(self.feature_names)
133
134         def to_dict(self) -> dict:
135             return {

```

```

136         "name": self.name,
137         "segmenter": self.segmenter,
138         "config_overrides": self.config_overrides,
139         "cue_flags": self.cue_flags,
140         "min_crossings": self.min_crossings,
141         "min_players": self.min_players,
142         "classifier_model": self.classifier_model,
143         "feature_names": self.feature_names,
144         "threshold": self.threshold,
145         "merge_gap": self.merge_gap,
146         "tune_threshold": self.tune_threshold,
147         "calibrate": self.calibrate,
148     }
149
150     @classmethod

```

7. assistant (2026-07-04T00:48:42.348Z)

Now let me search for the `_is_fusion_schema` function:

8. user (2026-07-04T00:48:44.893Z)

```

187: def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
224:     corpus-growth path persists) are SKIPPED, not crashed on (see ``_is_fusion_schema``), so
a
248:     if not _is_fusion_schema(cands):

```

9. user (2026-07-04T00:48:46.922Z)

```

180
181     name: str
182     intervals: List[Interval]
183     features: Dict[str, List[float]] = field(default_factory=dict)
184     gts: Optional[List[Interval]] = None
185
186
187     def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
188         """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
189         read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
190         ``fusion_audio.augment`` and older JSONs predate it).
191
192         The velocity-windowing / distill corpus-growth path persists a LEANER
193         ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts)
for clips
194         that never went through fusion feature extraction. The loaders that build cue-rich
195         ``VideoCandidates`` from these files ? ``ablation.load_videos``,
``fusion_compare.load_videos``
196         and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
``c["shuttle"]``,
197         so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
198         An empty candidate list is not a fusion file either.
199
200         Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
201         each landed an independent copy)."""
202         return bool(candidates) and all(
203             "shuttle" in c and "person" in c for c in candidates
204         )
205
206
207     def load_golden_videos(
208         features_dir: str,

```

```

209         *,
210         shuttle_columns: Sequence[str],
211         person_columns: Sequence[str],
212         audio_columns: Sequence[str] = (),
213         audio_policy: str = "omit",
214         skip_logger: logging.Logger = logger,
215         skip_message: str,
216         no_audio_error: str = "{path} has no audio features",
217     ) -> List["VideoCandidates"]:
218         """Shared golden-feature corpus loader behind the three structurally-identical
loaders
219         (`ablation.load_videos` / `fusion_compare.load_videos` /
`fusion_audio.load_videos_with_audio`).
220
221         Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
222         ``VideoCandidates`` (intervals + the caller-supplied feature columns + golden gts).
Leaner
223         velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
224         corpus-growth path persists) are SKIPPED, not crashed on (see
``_is_fusion_schema``), so a
225         MIXED corpus loads the fusion subset cleanly.
226
227         The column lists are ALWAYS supplied by the caller ? this loader never hardcodes
columns. The
228         three callers differ ONLY in:
229
230         - ``audio_policy`` ? how the audio block is handled, checked BEFORE reading any
audio column
231         so a windowing-only file is skipped before either branch applies:
232         * ``"omit"`` ? no audio columns are loaded (``audio_columns``
ignored);
233         * ``"tolerate-missing"`` ? audio columns load, a missing block defaults them
to 0.0;
234         * ``"require"`` ? a fusion file lacking audio raises ``SystemExit``
(via
235         ``no_audio_error`` formatted with ``path``), mirroring "run --augment"
first".
236         - ``skip_logger`` / ``skip_message`` ? the per-caller logger + ``%s``-templated
info line
237         emitted when a windowing-only file is skipped (kept byte-identical per caller).
238         """
239         if audio_policy not in ("omit", "tolerate-missing", "require"):
240             raise ValueError(
241                 f"audio_policy must be 'omit'/'tolerate-missing'/'require', got
{audio_policy!r}"
242             )
243         videos: List[VideoCandidates] = []
244         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
245             with open(path, encoding="utf-8") as f:
246                 d = json.load(f)
247                 cands = d["candidates"]
248                 if not _is_fusion_schema(cands):
249                     skip_logger.info(skip_message, os.path.basename(path))
250                     continue
251                 if audio_policy == "require" and "audio" not in cands[0]:
252                     raise SystemExit(no_audio_error.format(path=path))
253                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
254                 features: Dict[str, List[float]] = {}
255                 for fn in shuttle_columns:
256                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
257                 for fn in person_columns:
258                     features[fn] = [float(c["person"][fn]) for c in cands]
259                 if audio_policy == "tolerate-missing":

```

Perfect. Now I need to check which loaders actually use `_is_fusion_schema`, and verify what happens in `run()` with empty candidates:

11. user (2026-07-04T00:48:50.671Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/ablation.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/fusion_compare.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/fusion_audio.py
```

12. user (2026-07-04T00:48:53.229Z)

```
def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
```

13. user (2026-07-04T00:48:56.582Z)

```
return experiment.load_golden_videos(
    features_dir,
```

14. user (2026-07-04T00:48:59.137Z)

```
return experiment.load_golden_videos(
    features_dir,
    shuttle_columns=SHUTTLE_FEATURE_NAMES,
    person_columns=PERSON_FEATURE_NAMES,
    audio_policy="omit",
```

15. user (2026-07-04T00:49:01.620Z)

```
return experiment.load_golden_videos(
    features_dir,
    shuttle_columns=SHUTTLE_FEATURE_NAMES,
    person_columns=PERSON_FEATURE_NAMES,
    audio_columns=af.AUDIO_FEATURE_NAMES,
```

16. assistant (2026-07-04T00:49:04.279Z)

Good! All three loaders use `experiment.load_golden_videos`, which uses `_is_fusion_schema`. Now let me verify the critical logic in `run()` and `extract_video`:

17. user (2026-07-04T00:49:05.563Z)

```
316         for vi, spec in enumerate(specs, 1):
317             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
318             existing = os.path.isfile(out)
319             existing_full = existing and is_full_fusion_feature_file(out)
320             # Resumable: skip a video whose full-fusion JSON already exists. A leaner
321             # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
322             if missing_fusion_only and not fresh and existing_full:
323                 print(
324                     f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325                     f"features already present -> {out}",
326                     flush=True,
327                 )
328                 written.append(out)
329                 continue
330             if not fresh and existing and existing_full:
331                 print(
332                     f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
333 "
```

```

333         f"-> {out} (use --fresh to redo)",
334         flush=True,
335     )
336     written.append(out)
337     continue
338 if not fresh and existing and not existing_full:
339     print(
340         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341         "is not full-fusion schema; re-extracting...",
342         flush=True,
343     )
344     print(
345         f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",

```

18. assistant (2026-07-04T00:49:07.446Z)

Now let me check the extract_video write logic to see if it can produce empty candidates:

19. user (2026-07-04T00:49:07.746Z)

```

148 def extract_video(
149     spec: Dict[str, Any],
150     detector: Optional[Any] = None,
151     frame_skip: int = 3,
152     iou: float = 0.5,
153     log_every_sec: float = 15.0,
154     telemetry: Optional[Any] = None,
155     use_gpu: bool = True,
156     detector_model: Optional[str] = None,
157     max_gpu_temp_c: Optional[float] = None,
158 ) -> Dict[str, Any]:
159     """One golden video -> a feature record (shuttle + person features per candidate
window).
160
161     Returns ``{name, fps, frame_width,
candidates: [{start, end, shuttle {}, person {}, label}], gts}``.
162     `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
163     when None a real `PersonDetector` is built (loads torchvision on first use).
164     `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
165     from the detection progress callback so a hard kill leaves a forensic runlog.
166     """
167     fps, fw = float(spec["fps"]), float(spec["frame_width"])
168     traj = str(spec["trajectory"])
169     points = TrackNetRunner.parse_trajectory_csv(traj)
170     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
171     gts = load_golden_gts(str(spec["labels"]))
172     labels = label_candidates(intervals, gts, iou)
173     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
174     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
175     axis, net_px, _span = rally_gate.estimate_play_axis(points)
176     video_path = _derive_video_path(spec)
177     if not os.path.isfile(video_path):
178         raise FileNotFoundError(
179             f"{spec.get('name')}: source video not found for fusion extraction:
{video_path}"
180         )
181
182     if detector is None:
183         from backend.pipeline.detectors.person_detector import PersonDetector
184
185     indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}

```

```

186         if detector_model:
187             indexing_cfg["detector_model"] = detector_model
188             detector = PersonDetector({"indexing": indexing_cfg})
189
190         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192         n = len(intervals)
193         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194         print(
195             f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...",
196             flush=True,
197         )
198         t0 = time.monotonic()
199         state = {"last": t0}
200
201         def _progress(done: int, total: int) -> None:
202             rec = None
203             if telemetry is not None:
204                 rec = telemetry.snapshot(done, total, phase=spec["name"])
205             if max_gpu_temp_c is not None and rec is not None:
206                 gpu = rec.get("gpu") or {}
207                 temp = gpu.get("temp_c")
208                 if temp is not None and float(temp) >= max_gpu_temp_c:
209                     raise RuntimeError(
210                         f"{spec['name']}: GPU temperature {float(temp):.1f}C reached "
211                         f"--max-gpu-temp-c {max_gpu_temp_c:.1f}; aborting before thermal
shutdown"
212                     )
213             now = time.monotonic()
214             if now - state["last"] >= log_every_sec or done >= total:
215                 el = now - t0
216                 rate = done / el if el > 0 else 0.0
217                 eta = (total - done) / rate if rate > 0 else 0.0
218                 pct = 100 * done // total if total else 0
219                 print(
220                     f"[fusion_golden]   {spec['name']}: detect frame {done}/{total} ({pct}%)
"
221                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
222                     flush=True,
223                 )
224                 state["last"] = now
225
226         per_window = detector.detections_over_windows(
227             video_path, win_frames, frame_skip, progress_cb=_progress
228         )
229
230         candidates: List[Dict[str, Any]] = []
231         for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
232             det = per_window[i]
233             det_fw = det.get("frame_width") or fw
234             det_fh = det.get("frame_height") or 0.0
235             # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
236             # normalisation fusion_features uses (x/width, y/height).
237             if axis == "x":
238                 net_norm = net_px / det_fw if det_fw > 0 else 0.5
239             else:
240                 net_norm = net_px / det_fh if det_fh > 0 else 0.5
241             sf, ef = win_frames[i]
242             shuttle_in = [p for p in points if sf <= p.frame <= ef]
243             person = ff.compute_person_features(
244                 det["frames"],
245                 shuttle_in,
246                 det_fw,

```



```

247         det_fh,
248         court_polygon=None,
249         net=(axis, net_norm),
250     )
251     candidates.append(
252         {
253             "start": float(s),
254             "end": float(e),
255             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
256             "person": {k: float(v) for k, v in person.items()},
257             "label": int(labels[i]),
258         }
259     )
260     return {
261         "name": spec["name"],
262         "fps": fps,
263         "frame_width": fw,
264         "candidates": candidates,
265         "gts": [[float(a), float(b)] for a, b in gts],
266     }
267

```

20. assistant (2026-07-04T00:49:09.591Z)

Perfect. Now let me verify what happens on line 170 when intervals is empty:

21. user (2026-07-04T00:49:11.180Z)

```

def build_candidates(
    trajectory_csv: str,
    fps: float,
    frame_width: float,
    proposal: Tuple[float, float, float] = PROPOSAL,
) -> Tuple[List[Interval], List[List[float]]]:
    """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature rows)."""
    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
    vt, mg, mwd = proposal
    wins = TrackNetRunner.trajectory_to_action_windows(
        points,
        fps=fps,
        frame_width=frame_width,
        velocity_thresh=vt,
        merge_gap=mg,
        min_window_duration=mwd,
    )
    intervals = [(w["start"], w["end"]) for w in wins]
    gated = rally_gate.gate_windows(
        points, intervals, fps, min_crossings=0
    ) # for net-crossings

```

22. assistant (2026-07-04T00:49:16.825Z)

So `build\_candidates` can return an empty list of intervals if the video has no detected action windows. In that case, `extract\_video` will write `candidates: []`. Let me trace through the `run()` logic:

Looking at lines 316-337 in the PR file:

- Line 319: `existing\_full = existing and is\_full\_fusion\_feature\_file(out)`
- If a file exists with `candidates: []`, the new `is\_full\_fusion\_feature\_file()` function (line 58-59) returns `**True**`
- Line 330-337: If `existing\_full` is True, it skips re-extraction: `written.append(out); continue``

But now check what happens in the downstream loaders. They all use

`experiment.load\_golden\_videos()` which calls `\_is\_fusion\_schema()`:

From experiment.py lines 202-204:

```
```python
return bool(candidates) and all(
 "shuttle" in c and "person" in c for c in candidates
)
```
```

For `candidates: []`:

- `bool([])` = **False**
- So `\_is\_fusion\_schema([])` returns **False**
- At line 248-250, the video is **skipped** (not loaded)

Let me verify there's actually a write happening in run():

23. user (2026-07-04T00:49:18.211Z)

```
362         if telem is not None:
363             telem.event("error", spec["name"], err=str(e))
364         raise
365         n_pos = sum(c["label"] for c in data["candidates"])
366         with open(out, "w", encoding="utf-8") as f:
367             json.dump(data, f, indent=2)
368         if telem is not None:
369             telem.event(
370                 "video_done",
371                 spec["name"],
372                 windows=len(data["candidates"]),
373                 positives=n_pos,
374             )
375         print(
376             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
377             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
378             f"-> {out}",
379             flush=True,
380         )
381         written.append(out)
382     return written
383
384
385     def main() -> None:
386         ap = argparse.ArgumentParser(
```

24. assistant (2026-07-04T00:49:26.724Z)

Perfect! Lines 366-367 show the JSON is always written, regardless of whether candidates is empty. Now let me create a summary document of the verification:

****CONFIRMED****

The correctness impact is real and demonstrated by two contradictory schema checks:

****The Disagreement:****

- ****PR's `\_is\_full\_fusion\_feature\_file(path)` line 58-59:**** `if not candidates: return True` ? an empty-candidates JSON is treated as a "full-fusion" file
- ****Existing `\_is\_fusion\_schema(candidates)` line 202-204 (experiment.py):**** `return bool(candidates) and all(...)` ? an empty-candidates list returns ****False****

****The Corpus Gap (lines quoted):****

1. ****Resume logic skips re-extraction:**** fusion_golden.py lines 330-337: when `existing\_full` is True (which happens for `candidates: []` per line 319), the code prints `"already extracted ->

`{out}"` and calls written.append(out); continue`, skipping extract_video entirely.`

2. **Downstream loaders exclude the same video:** `experiment.py` lines 248-250 uses `_is_fusion_schema(cands)`` to check; for `candidates: []`` this returns `False`, so the video is skipped with `skip_logger.info(skip_message, ...)``. The loaders (`ablation.py`, `fusion_compare.py`, `fusion_audio.py`) all call `experiment.load_golden_videos()`` which uses this check.

3. **Empty-candidates files are writable:** `extract_video` (line 230-259) builds candidates from intervals; if `distill_local.build_candidates()`` returns an empty intervals list (reachable for videos with no detected rally windows), the for-loop at line 231 produces zero candidates, which are written to JSON at lines 366-367.

The Silent Gap: A video that extracts to 0 candidate windows gets (a) cached by `run()` as "already full-fusion" and never re-extracted, yet (b) silently excluded from every fusion analysis (`ablation`, `fusion_compare`, `fusion_audio`) because the loaders skip empty-candidate files. This creates a corpus inconsistency: `fusion_golden.run()` writes it as "done" but no downstream fusion code sees it.

One-line fix:

`_is_full_fusion_feature_file`` should reuse `experiment._is_fusion_schema(candidates)`` and drop the `if not candidates: return True`` special case, or align the two functions' empty-candidate semantics.